

```
"""
```

```
Market News Scanner
```

```
=====
```

```
Claude Haiku סורק מקורות חדשות (כלכליות ישראליות + בינלאומיות), מסנן עם  
לפי רלוונטיות לשוק ההון, ושולח התראה בטלגרם על כל פריט רלוונטי חדש.
```

```
לא כההליך שרץ לנצח, ('וכו cron, GitHub Actions) מתוזמן job-מיועד לרוץ כ  
state. קודם -> בודקת מה חדש -> מסננת -> שולחת -> שומרת state כל הרצה: קוראת.  
"""
```

```
import os  
import re  
import json  
import sys  
from datetime import datetime, timezone
```

```
import feedparser  
import requests  
from anthropic import Anthropic
```

```
# -----  
# הגדרות  
# -----
```

```
STATE_FILE = "seen_ids.json"  
MAX_SEEN_PER_SOURCE = 300 # הים לזכור לכל מקור (כדי שהקובץ לא יגדל לנצח)  
MAX_ITEMS_PER_SOURCE_PER_RUN = 8 # פני הצפה בהרצה הראשונה / מקור עם הרבה פוסטים  
CLAUDE_MODEL = "claude-haiku-4-5-20251001"
```

```
# אפשר להוסיף/להסיר כרצונך. RSS מקורות.  
# ל אתרים ישראליים משתנות מדי פעם - מומלץ לבדוק שהן עדיין חיות RSS שים לב: כתובות  
# (אפשר לבדוק כל כתובת פשוט על ידי פתיחה בדפדפן).
```

```
RSS_SOURCES = {  
    "Globes": "https://www.globes.co.il/websevice/rss/rssfeeder.aspx/FeederNode?  
    "TheMarker_Economy": "https://www.themarker.com/cmlink/1.144",  
    "CNBC_Markets": "https://www.cnbc.com/id/20910258/device/rss/rss.html",  
    "Reuters_Business": "https://feeds.reuters.com/reuters/businessNews",  
    "GoogleNews_Trump_Markets": (  
        "https://news.google.com/rss/search?q=Trump%20(stock%20OR%20market%20OR%2  
        "%20when:1d&hl=en-US&gl=US&ceid=US:en"  
    ),  
    "GoogleNews_Fed_Macro": (  
        "https://news.google.com/rss/search?q=(%22Federal%20Reserve%22%20OR%20%22  
        "%20when:1d&hl=en-US&gl=US&ceid=US:en"
```

```
),  
}
```

```
CLASSIFY_PROMPT = """לזיהוי אירועים שעשויים להשפיע באופן מהותי וישיר על שוק ההון  
(מניה ספציפית, מדד, סחורה, מטבע, ריבית).
```

```
כותרת: {title}  
תקציר/טקסט: {summary}  
מקור: {source}
```

```
תקני, ללא טקסט נוסף, ללא ```), בפורמט הזה JSON החזר אך ורק  
{"relevant": true/false, "tickers": ["SYMBOL"], "urgency": "high/medium/low", "r
```

```
הנחיות:
```

```
- relevant=true ות סבירה ממשית שהאירוע יזיז מחיר של מניה/מדד/סחורה/ריבית ספציפיים  
- relevant=false. - חדשות כלליות, ניתוחים, דעות, או דברים שכבר ידועים/מתומחרים  
- urgency=high (עסקה, רגולציה פתאומית, הצהרה ישירה על חברה/שוק, נתון מאקרו מפתיע)  
- tickers: אם ידוע השם/הסימול של החברה - ציין, אחרת מערך ריק.  
"""
```

```
def log(msg: str) -> None:  
    ts = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")  
    print(f"[{ts}] {msg}", flush=True)
```

```
# -----  
# State (כדי לא להתריע פעמיים על אותו פריט)  
# -----
```

```
def load_state() -> dict:  
    if not os.path.exists(STATE_FILE):  
        return {}  
    try:  
        with open(STATE_FILE, "r", encoding="utf-8") as f:  
            return json.load(f)  
    except Exception:  
        return {}
```

```
def save_state(state: dict) -> None:  
    with open(STATE_FILE, "w", encoding="utf-8") as f:  
        json.dump(state, f, ensure_ascii=False, indent=2)
```

```
def entry_id(entry) -> str:  
    return entry.get("id") or entry.get("link") or entry.get("title", "")
```

```

# -----
# ממשק Claude
# -----

def classify(client: Anthropic, title: str, summary: str, source: str) -> dict:
    prompt = CLASSIFY_PROMPT.format(title=title, summary=summary[:500], source=so
    try:
        resp = client.messages.create(
            model=CLAUDE_MODEL,
            max_tokens=300,
            messages=[{"role": "user", "content": prompt}],
        )
        text = resp.content[0].text.strip()
        text = re.sub(r"^```(json)?|```$", "", text, flags=re.MULTILINE).strip()
        data = json.loads(text)
        data.setdefault("relevant", False)
        return data
    except Exception as e:
        log(f" classify error: {e}")
        return {"relevant": False}

```

```

# -----
# טלגרם
# -----

```

```

def send_telegram(token: str, chat_id: str, text: str) -> None:
    url = f"https://api.telegram.org/bot{token}/sendMessage"
    try:
        r = requests.post(
            url,
            data={
                "chat_id": chat_id,
                "text": text,
                "parse_mode": "HTML",
                "disable_web_page_preview": "false",
            },
            timeout=15,
        )
        if r.status_code != 200:
            log(f" telegram error: {r.status_code} {r.text}")
    except Exception as e:
        log(f" telegram exception: {e}")

```

```

def format_alert(title: str, link: str, source: str, info: dict) -> str:
    urgency_emoji = {"high": "🔴", "medium": "🟡", "low": "⚪"}.get(info.get("urgency"), "🟡")
    tickers = ", ".join(info.get("tickers", []) or [])
    lines = [
        f"{urgency_emoji} <b>התראה חשובה</b>:",
        f"<b>{title}</b>",
    ]
    if tickers:
        lines.append(f"טיקרים: {tickers}")
    if info.get("reason_he"):
        lines.append(f"למה זה רלוונטי: {info['reason_he']}")
    lines.append(f"מקור: {source}")
    lines.append(link)
    return "\n".join(lines)

```

```

# -----
# ראשי
# -----

```

```

def main() -> None:
    telegram_token = os.environ.get("TELEGRAM_BOT_TOKEN")
    telegram_chat_id = os.environ.get("TELEGRAM_CHAT_ID")
    anthropic_key = os.environ.get("ANTHROPIC_API_KEY")

    missing = [
        name
        for name, val in [
            ("TELEGRAM_BOT_TOKEN", telegram_token),
            ("TELEGRAM_CHAT_ID", telegram_chat_id),
            ("ANTHROPIC_API_KEY", anthropic_key),
        ]
        if not val
    ]
    if missing:
        log(f"חסרים environment variables: {', '.join(missing)}. עוצר.")
        sys.exit(1)

    client = Anthropic(api_key=anthropic_key)
    state = load_state()
    first_run = len(state) == 0
    if first_run:
        log("'הרצה ראשונה - לא יישלחו התראות על תוכן קיים, רק נסמן אותו כ'נקרא'")

    alerts_sent = 0
    items_checked = 0

```

```

for source_name, feed_url in RSS_SOURCES.items():
    log(f"סורק מקור: {source_name}")
    seen_ids = set(state.get(source_name, []))

    try:
        feed = feedparser.parse(feed_url)
    except Exception as e:
        log(f"שגיאה בקריאת הפיד: {e}")
        continue

    if getattr(feed, "bozo", False) and not feed.entries:
        log(f"דלג (bozo). הפיד לא תקין או ריק")
        continue

    new_entries = []
    for entry in feed.entries:
        eid = entry_id(entry)
        if eid and eid not in seen_ids:
            new_entries.append(entry)

    # להגביל כמות בהרצה אחת (הגונה מהצפה)
    new_entries = new_entries[:MAX_ITEMS_PER_SOURCE_PER_RUN]

    for entry in new_entries:
        eid = entry_id(entry)
        title = entry.get("title", "")
        summary = entry.get("summary", "") or entry.get("description", "")
        link = entry.get("link", "")

        seen_ids.add(eid)

    if not first_run:
        items_checked += 1
        info = classify(client, title, summary, source_name)
        if info.get("relevant") and info.get("urgency") in ("high", "medi
            text = format_alert(title, link, source_name, info)
            send_telegram(telegram_token, telegram_chat_id, text)
            alerts_sent += 1
            log(f"✓ נשלחה התראה: {title[:80]}")

    # שמירה על גודל סביר ל state
    state[source_name] = list(seen_ids)[-MAX_SEEN_PER_SOURCE:]

save_state(state)
log(f"התראות {alerts_sent} פריטים חדשים, נשלחו {items_checked} סיוס. נבדקו")

```

```
if __name__ == "__main__":  
    main()
```